Replay Court Specification v0.2

> Extends v0.1. Defines MutationProposalV1, AuthorityContextV1, and the first replay test vector.

---

7. MutationProposalV1

```
pub struct MutationProposalV1 {
    pub schema_version: u16,
    pub proposal_id: [u8; 32],
    pub proposer_id: [u8; 32],
    pub target_agent_id: [u8; 32],
    pub mutation_type: u32,
    pub payload_hash: [u8; 32],
    pub payload: Vec<u8>,
    pub semantic_version_required: u32,
    pub codec_version_required: u32,
    pub nonce: u64,
    pub created_at: u64,
    pub expires_at: u64,
    pub proposer_signature: Vec<u8>,
}
```

Canonical proposal hash (used as `mutation_hash` in `StateTransitionV1`):

```
proposal_id =
  keccak256(
    abi.encode(
      schema_version,
      proposer_id,
      target_agent_id,
      mutation_type,
      payload_hash,
      semantic_version_required,
      codec_version_required,
      nonce,
      created_at,
      expires_at
    )
```

```
)
```

`payload_hash = keccak256(payload)`

`proposer_signature` signs `proposal_id`. Not included in `proposal_id`.

Mutation Types

Code	Name	Description
`0x0001`	`GENOME_PATCH`	Structured patch to one or more genome fields
`0x0002`	`GENOME_REPLACE`	Full genome replacement (requires elevated authority)
`0x0003`	`CAPABILITY_GRANT`	Adds a capability flag to the agent
`0x0004`	`CAPABILITY_REVOKE`	Removes a capability flag
`0x0005`	`ATTESTATION_ATTACH`	Attaches a signed attestation to the agent record
`0x0006`	`LINEAGE_FORK`	Forks agent into a new lineage (immutable after)
`0x00FF`	`TOMBSTONE`	Terminates agent; no further mutations allowed

Unknown mutation types → `CourtOutcome::InvalidSemanticVersion`.

Expiry

If `created_at` is outside the epoch window or `expires_at < transition.created_at`, the proposal is expired.

Expired proposals return `CourtOutcome::InvalidAuthority`.

The Replay Court does **not** reject expired proposals on historical replay — it records them via `AuthorityReview.expired_attestation_flags`.

---

8. AuthorityContextV1

```
pub struct AuthorityContextV1 {
    pub schema_version: u16,
    pub context_id: [u8; 32],
    pub governor_id: [u8; 32],
    pub authority_chain: Vec<AuthorityNode>,
```

```
    pub policy_hash: [u8; 32],
    pub policy: Vec<u8>,
    pub attestations: Vec<AttestationV1>,
    pub epoch: u64,
    pub created_at: u64,
}

pub struct AuthorityNode {
    pub node_id: [u8; 32],
    pub role: u32,
    pub delegation_depth: u8,
    pub node_signature: Vec<u8>,
}

pub struct AttestationV1 {
    pub attestation_id: [u8; 32],
    pub attester_id: [u8; 32],
    pub subject_id: [u8; 32],
    pub claim_hash: [u8; 32],
    pub valid_from: u64,
    pub valid_until: u64,
    pub attester_signature: Vec<u8>,
}
```

Canonical context hash (used as `authority_context_hash` in `StateTransitionV1`):

```
context_id =
  keccak256(
    abi.encode(
      schema_version,
      governor_id,
      hash_authority_chain(authority_chain),
      policy_hash,
      hash_attestations(attestations),
      epoch,
      created_at
    )
  )
```

Where:

```
hash_authority_chain(chain) =
  keccak256(abi.encode(
    for each node: (node_id, role, delegation_depth)
    // node_signature excluded
```

```
  ))

hash_attestations(attestations) =
  keccak256(abi.encode(
    for each attestation: (attestation_id, attester_id, subject_id, claim_hash, v
    // attester_signature excluded
  ))
```

Signatures are excluded from content hashes. Each node and attester signs its own `node_id` / `attestation_id`.

Authority Roles

Code	Role	Max Delegation Depth
`0x01`	`ROOT_GOVERNOR`	0 (self-signing only)
`0x02`	`DOMAIN_GOVERNOR`	1
`0x03`	`AGENT_SUPERVISOR`	2
`0x04`	`PROPOSER`	3

Delegation depth must decrease monotonically down the chain. Any break → `CourtOutcome::InvalidAuthority`.

Policy

`policy` is a canonical ABI-encoded byte array interpreted by the mutation engine WASM.

```
policy_hash = keccak256(policy)
```

The Replay Court does not interpret policy directly — it passes it to `apply_mutation` as part of `context`. Policy validity is the mutation engine's responsibility.

---

9. Hash Function Definitions

For cross-implementation consistency, the following canonical definitions apply:

```
state_hash(state: AgentGenomeStateV2) -> [u8; 32]
  = keccak256(canonical_abi_encode(state))

hash_mutation(proposal: MutationProposalV1) -> [u8; 32]
```

```
  = proposal.proposal_id

hash_authority_context(ctx: AuthorityContextV1) -> [u8; 32]
  = ctx.context_id

hash_verdict(verdict: VerdictV1) -> [u8; 32]
  = keccak256(canonical_abi_encode(verdict.outcome, verdict.reason_code, verdict.
```

`canonical_abi_encode` is strict ABI encoding: no padding variation, field order is struct declaration order, no optional omission.

---

10. First Replay Test Vector

This is the normative test vector. Any Replay Court implementation must produce identical results.

10.1 Inputs

domain_separator (32 bytes, hex):

```
0000000000000000000000000000000000000000000000000000000000000001
```

pre_state (`AgentGenomeStateV2` , abbreviated):

```
{
  "agent_id": "aaaa...0001",
  "version": 1,
  "capability_flags": 0,
  "genome_fields": {},
  "lineage_root": "0000...0000"
}
```

pre_state_hash:

```
keccak256(canonical_abi_encode(pre_state))
= <implementation must compute and assert equality>
```

proposal (`MutationProposalV1`):

```
{
  "schema_version": 1,
  "proposer_id": "bbbb...0002",
```

```
  "target_agent_id": "aaaa...0001",
  "mutation_type": 3,
  "payload": "<abi-encoded CAPABILITY_GRANT: flag=0x0001>",
  "semantic_version_required": 1,
  "codec_version_required": 1,
  "nonce": 1,
  "created_at": 1700000000,
  "expires_at": 1800000000
}
```

authority_context (`AuthorityContextV1`):

```
{
  "schema_version": 1,
  "governor_id": "cccc...0003",
  "authority_chain": [
    {
      "node_id": "cccc...0003",
      "role": 1,
      "delegation_depth": 0
    },
    {
      "node_id": "bbbb...0002",
      "role": 4,
      "delegation_depth": 1
    }
  ],
  "policy": "<allows CAPABILITY_GRANT of flag 0x0001 to any agent>",
  "attestations": [],
  "epoch": 1,
  "created_at": 1700000000
}
```

mutation_engine_wasm: `mutation_engine_v1_test.wasm`

The reference WASM must implement `apply_mutation` such that for this input it:

1. Decodes the pre_state
2. Verifies the policy permits `CAPABILITY_GRANT` of `flag=0x0001`
3. Sets `capability_flags |= 0x0001`
4. Returns canonical ABI-encoded `AgentGenomeStateV2`

10.2 Expected Outputs

```
mutation_engine_hash = keccak256(wasm_bytes_of_reference_engine)

authority_review = AuthorityReview {
  valid_at_execution: true,
  valid_under_current_law: true,
  decayed_reputation_score: 0,
  expired_attestation_flags: 0,
  review_required: false
}

verdict = VerdictV1 {
  outcome: Allow,
  reason_code: 0,
  flags: 0
}

post_state = AgentGenomeStateV2 {
  ...pre_state,
  capability_flags: 1
}

court_outcome = CourtOutcome::TransitionVerified
```

10.3 Failure Cases (Must Also Pass)

Perturbation	Expected Outcome
Flip one byte of `mutation_engine_wasm`	`InvalidMutationEngine`
Set `expires_at = created_at - 1`	`InvalidAuthority`
Set `delegation_depth[1] = 0` (non-monotonic)	`InvalidAuthority`
Set `mutation_type = 0x00FF` (TOMBSTONE, policy disallows)	`TransitionChallenged`
Corrupt `governor_signature`	`InvalidSignature`
Set `post_state_hash` to all zeros	`InvalidPostState`
Set `pre_state_hash` to wrong value	assertion failure before court outcome

Each failure case must produce exactly the listed `CourtOutcome` and no state change.

11. Next Steps

1. Implement `AgentGenomeStateV2` canonical ABI schema
2. Build reference `mutation_engine_v1_test.wasm` with the test vector behavior
3. Compute and publish concrete byte values for all hashes in §10.2
4. Write conformance test harness that runs §10.3 failure cases automatically
5. Define `VerdictV1` struct in full ABI style (currently abbreviated)
6. Define the Governor key scheme (Ed25519 or secp256k1 — pick one and lock it)